

Making AI Coding Assistants Cheaper Without Making Them Dumber

Benjamin Wynia

July 4, 2026

CONTENTS

What does this project do?	
How does it work?	
What does “deterministic” mean, and why does it matter?	
How did we test it?	
What were the results?	
What are the expected cost savings?	
Should we implement this?	
What would this look like for an engineer using VS Code + Copilot?	

Making AI Coding Assistants Cheaper Without Making Them Dumber

A plain-language report on the context-compression project

July 4, 2026

What does this project do?

When you chat with an AI coding assistant, the assistant doesn't actually "remember" your conversation. Every single time it responds, the entire conversation so far — every question, every answer, every file it looked at — gets sent back to the AI company's servers, and you pay for every word of it.

That transcript grows fast. By the end of a long working session, each new question might drag along hundreds of pages of history. Most of that history is stale: files the assistant read an hour ago, error messages that were already fixed, output from commands that no longer matter. You're paying, over and over, to re-send things the assistant no longer needs.

This project (called **ctxc**) is a slimming layer that sits between your coding tool and the AI. Before each message goes out, it trims the transcript down to a fixed size — keeping what matters, shrinking what doesn't — so the AI gets a shorter, cheaper conversation that still contains everything it needs to do the job.

How does it work?

Think of the conversation as a long meeting transcript. ctxc edits it the way a good assistant would prepare a briefing:

1. **The original instructions are sacred.** Your first request and the ground rules are always kept word-for-word, never summarized. The AI never loses sight of what it was asked to do.
2. **Recent conversation is kept in full.** The last several exchanges are untouched, because that's what the AI is actively working with.
3. **Old middle material gets condensed.** Older file contents and command outputs are shortened: the beginning and end are kept, the bulky middle is replaced with a marker saying "content trimmed here." The AI can always re-open a file if it truly needs it again.

4. **A one-page “minutes of the meeting” is inserted.** Everything that was trimmed away is represented by a single summary note near the top, so the AI knows what happened earlier even though the full text is gone.
5. **Important-looking details are protected.** Things that look like decisions, code names, version numbers, or notes-to-self are recognized and kept even when the text around them is trimmed.

What does “deterministic” mean, and why does it matter?

Most approaches to this problem ask *another AI* to write the summaries. That is slow, costs extra money, and — worse — gives different results every time, so you can never fully test it or predict what it will do.

ctxc is **deterministic**: it follows fixed rules, like a recipe. Given the same conversation, it produces exactly the same trimmed version, every time, in about a millisecond, with no extra AI calls. We proved this directly: we ran the same real conversation through it three times and got byte-for-byte identical output.

Deterministic also means **testable**. Every trimmed conversation is checked against hard safety rules: the size limit is never exceeded, the original instructions are never altered, and the structure the AI expects is never broken. Across every test we ran — thousands of automated checks and a full live experiment — those rules were never violated once.

How did we test it?

In stages, each one harder than the last:

- **Bench tests.** 119 automated tests covering every rule and edge case.
- **Real transcript replay.** We took a real GitHub Copilot working session from this computer and replayed it through the compressor, verifying every safety rule at every step and confirming identical results across runs.
- **Memory checks.** We planted specific facts early in long conversations, compressed them, and checked which facts survived. Prominent facts (decisions, codes, flagged notes) survived 100% of the time. Quietly-worded facts buried mid-conversation can be lost under aggressive trimming — this is the method’s known weak spot, and it’s measurable and repeatable.

- **The main event: a 50-task head-to-head trial.** We had an AI agent fix 50 real, randomly-chosen bugs from well-known open-source projects (Django, scikit-learn, matplotlib, and nine others) — twice. Once normally, and once with ctxc silently compressing everything to roughly **half size**. Whether each bug was truly fixed was judged by the benchmark’s official test harness, not by us. The task list was locked in before any results were seen, and the compression setting was chosen by a rule written down in advance.

What were the results?

	With compression	Without compression
Bugs officially fixed	14 of 50	13 of 50
Words sent to the AI	80% fewer	—
Actual dollars spent	\$2.20	\$3.02 (27% saved)
Compression failures	0	—

The compressed agent fixed *one more* bug than the normal one. That small edge is luck, not proof of improvement — but it is strong evidence of the thing that matters: **cutting the conversation in half did not make the assistant worse at its job**. On the tasks where the two disagreed, wins split almost evenly (5 to 4), which is what coin-flip noise looks like.

One honest caution from the details: on a few tasks, the compressed agent took noticeably more steps to finish, and a longer wander sometimes ended in a wrong answer. The effect cut both ways and was too small to measure reliably, but it’s the right thing to watch in any larger rollout.

What are the expected cost savings?

The honest answer: **it depends on how long the conversations get**.

- The “80% fewer words” number shrinks to **27% actual dollar savings**, because AI providers already give a big discount for re-sent text they recognize from the previous message (their own caching). Compression partially disrupts that discount, and our accounting includes that penalty.
- **Short conversations: little or no savings.** An earlier small trial showed that on brief sessions, compression can even cost slightly more than it saves. The tool detects this and can simply stay out of the way.
- **Medium conversations (our trial): ~27% real savings.**

- **Long conversations — the way Copilot is actually used all day — should save the most, but we haven't proven it yet.** Real all-day sessions grow far larger than our test tasks did, and the bigger the transcript, the bigger the win. Measuring this on real traffic is the logical next step.

There's also a benefit money doesn't capture: conversations that stay under the size limit **don't die**. Without compression, a long session eventually hits the AI's hard ceiling and fails or forgets; with it, the session keeps going indefinitely.

Should we implement this?

A qualified yes — through a careful next step, not a big-bang switch.

What's been established: the mechanism is safe (zero rule violations, zero failures across every test), quality was statistically unharmed at half-size compression, and real savings of ~27% showed up in honest, cache-aware accounting on medium-length sessions.

What's *not* yet established: how it behaves on genuine all-day engineering sessions (our tasks were shorter than real life), and where the breaking point is if you compress much harder than 2-to-1.

The recommended path is **shadow mode**: run ctxc silently alongside real traffic, compressing every conversation but *sending the original*, while recording what it would have saved and what it would have trimmed. That produces a precise, zero-risk forecast of savings on your actual usage before a single real conversation is altered.

What would this look like for an engineer using VS Code + Copilot?

For the engineer: nothing changes. No new buttons, no new habits. They write code, ask Copilot questions, let it fix bugs — exactly as today.

Behind the scenes, the pieces work like this:

1. Copilot's requests, instead of going straight to the AI provider, pass through the ctxc relay (a small service IT runs; Copilot supports pointing at such a gateway in enterprise setups).
2. The relay recognizes each ongoing session, trims the transcript to the size budget using the rules above, and forwards the slimmed version. This adds about a millisecond — imperceptible.

3. The answer flows back unchanged. The engineer sees exactly what they would have seen; the meter simply ran on half as many words.

Day to day, an engineer might notice exactly two things, both good: very long sessions no longer bog down or hit “conversation too long” failures, and — on metered plans — the same work burns noticeably less quota.

The realistic rollout: measure in shadow mode for a couple of weeks, turn compression on for a pilot group at the same gentle 2-to-1 level we tested, compare the groups’ outcomes and bills, then expand.

Everything above is reproducible from this repository: the compressor, the test suites, the trial scripts, and the raw results. Total cost of the entire 100-run graded experiment: about \$6 in AI usage.